90-DAY JOURNEY

DAY 06/90

AI SECURITY & RED TEAMING

Day 6 — Authentication & Authorization

Level: Beginner | **Study time: 3 hours** | **3 parts + hands-on lab**

Phase 1 · Foundation · For students, working professionals & freelancers

IN THIS GUIDE

PART 1 Who Are You? — Authentication Basics

PART 2 What Can You Do? — Authorization Models & AI Risks

PART 3 Defenses & Secure Design

PART 4 Hands-on Practical Worksheet

OBJECTIVE

Understand how apps prove who a user is (authentication) and decide what that user may access (authorization), why AI apps make authorization even more important, and test both on your own local API using two test accounts.

Educational use only — test only with authorization

PART 1

Who Are You? — Authentication Basics

Hour	Focus	What you do today
HR 1	Learn	Authentication methods, sessions, tokens and API keys (Part 1) and authorization models (Part 2).
HR 2	Build	Add token-based login with two users to your Day 4 API (Part 4, Tasks 1–2).
HR 3	Red Team	Test cross-user access with two accounts, fix the missing ownership check, retest (Tasks 3–5). Last 10 min: write notes in `DAY-XX.md`.

1. Authentication vs authorization

	Authentication (AuthN)	Authorization (AuthZ)
Question	**Who are you?**	**What are you allowed to do?**
Example	Login with email + password, OTP, Google sign-in.	Student can read own marks; only staff can edit marks.
Typical failure	Weak passwords, no rate limit on login, tokens that never expire.	User sees another user's data; normal user reaches admin features.
HTTP code when missing	`401 Unauthorized`	`403 Forbidden` (or `404` to avoid revealing existence)

> **! KEY POINT**
> Most serious real-world API findings are **authorization** failures, not authentication failures. Being logged in does not mean being allowed.

2. How apps remember you after login

Method	How it works	Security notes
Session cookie	Server stores session data; browser keeps a random session id in a cookie.	Cookie flags `HttpOnly`, `Secure`, `SameSite`. Session must be invalidated on logout.
Bearer token (e.g. JWT)	Client sends `Authorization: Bearer <token>` on every request.	Server must verify the signature and expiry. Store tokens carefully on the client.
API key	Long secret string identifying an app or account.	Keep server-side only, scope to minimum permissions, rotate, never ship in frontend / mobile code.
OAuth / SSO	A trusted identity provider (Google, Microsoft) signs you in.	Validate redirect URIs and tokens properly; request minimal scopes.

3. What a JWT looks like

A **JSON Web Token** has three base64url parts separated by dots: **header . payload . signature**. The payload is only encoded, **not encrypted** — anyone holding the token can read it.

```
● JWT structure

eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJ1MSIsInJvbGUiOiJ1c2VyIiwiZXhwIjoxNzYxMDAwMDAwfQ . 3q2-...
      header                          payload                                          signature
{"alg":"HS256"}          {"sub":"u1","role":"user","exp":1761000000}        proves it was not changed
```

! REMEMBER

Never put secrets or sensitive personal data in a JWT payload. And a valid token only proves identity — the server must still check permissions for **every** resource.

4. Login protections to look for

01 Rate limiting / lockout on login attempts	**02** Multi-factor authentication (MFA)
03 Same generic error for wrong user or wrong password	**04** Tokens expire and can be revoked
05 Password reset links are single-use and expire	**06** Logout really invalidates the session
07 Strong password hashing (bcrypt / argon2)	**08** API keys scoped and rotated

WHO ARE YOU? — HOW TODAY HELPS YOU

● STUDENT

Build login + roles into one of your own projects (college project, hackathon app). Understanding it from the builder side makes testing much easier.

● WORKING PRO

Ask: in our internal AI tools, who can see whose chats and documents? Simply mapping that is valuable for your team.

● FREELANCER

Authorization bugs are among the most common and highest-paid findings in real assessments and bug bounty programs. Always budget time for two-account testing.

PART 2

What Can You Do? — Authorization Models & AI Risks

1. Authorization models

Model	Idea	Example
RBAC — role based	Permissions attached to roles; users get roles.	`student`, `staff`, `admin`.
ABAC — attribute based	Rules use attributes of user, resource and context.	Staff may edit marks only for their own department.
Ownership checks	Resource belongs to a user; only the owner (or admin) can access it.	Only u1 can read conversation 1.
Least privilege	Give each user, service and agent the minimum access needed.	Chatbot service account can read FAQs, not HR records.

2. The common authorization failures (OWASP API Top 10)

Failure	What happens	AI app example
BOLA / IDOR (API1)	Changing an object id gives access to another user's object.	`GET /api/conversations/2` returns someone else's chat.
BOPLA (API3)	User can read or set object properties they should not.	Setting `"role": "admin"` or `"owner"` in a request body.
BFLA (API5)	Normal user can call privileged functions.	Normal user calls `PUT /api/admin/system-prompt`.
Missing auth on endpoint	Some endpoint forgot to require login.	`/api/export` works without a token.
Client-side only checks	UI hides a button, but the API still allows the action.	'Delete document' hidden for students, API still accepts it.

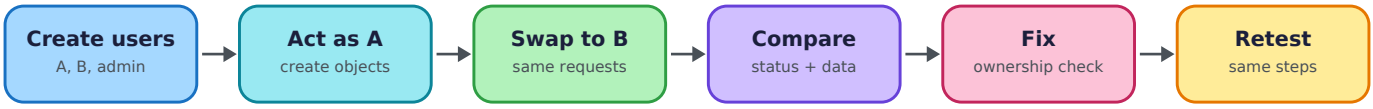
3. Why AI apps raise the stakes

In AI apps, the model often reads data and triggers actions **on behalf of the user**. If authorization is weak, the AI becomes a very convenient way to reach data the user should never see.

AI component	Authorization question
Chat history	Can a user list or open another user's conversations?
RAG documents	Are document permissions checked **before** content goes into the model's context? (Day 32)
Agent tools	Does each tool call run with the **user's** permissions, or with a powerful service account? (Day 43, 48)
Memory	Can one user's stored memory appear in another user's session? (Day 51)
Admin features	Can a normal user change system prompts, model settings or guardrails?

> **! KEY RULE**
>
> The LLM must **never** be the component that decides access. A system prompt saying "only show users their own data" is not a security control. The backend must check permissions in code, before data reaches the model.

4. Two-account testing method



01 Create (or get from the client) at least two normal accounts and, if relevant, one admin account.

02 As User A, create resources (chat, document, setting) and note every id.

03 Repeat User A's requests with User B's token / cookie, keeping A's ids.

04 Repeat admin-only requests with a normal user's token.

05 Record status codes and response bodies. Use harmless canary text so leaks are easy to prove.

06 Report, fix (or recommend fix), and retest the exact same steps.

> **! SCOPE RULE**
>
> Test only systems you own or have written permission to test: your own laptop, your own lab VM, intentionally vulnerable practice labs (PortSwigger, OverTheWire, Gandalf) or a client engagement with a signed scope.

PART 3

Defenses & Secure Design

1. Defense checklist

Area	Secure practice
Every request	Authenticate, then authorize **each** resource and action on the server. Deny by default.
Object access	Query with the owner in the filter, e.g. `WHERE id = ? AND owner = current_user`.
Input schemas	Never accept `owner`, `role` or `is_admin` from the client; set them on the server.
Admin functions	Separate routes with explicit role checks; log every admin action.
Tokens	Short expiry, verified signature with a fixed algorithm, revocation on logout / password change.
IDs	Random ids (UUIDs) reduce guessing — but are **not** a replacement for authorization checks.
AI data paths	Filter RAG documents and tool results by the user's permissions before they enter the model context.
Testing	Automated tests that try cross-user access for every endpoint (you will build one on Day 22).

▸ **EXAMPLE**

A college chatbot lets students ask about their attendance. Secure design: the backend reads the student id from the verified session, fetches **only** that student's records, and gives just those to the model. The model never receives other students' data, so even a manipulated prompt cannot reveal it.

2. Secure pattern in code

● ownership check pattern

```
def get_conversation(cid: int, user: User = Depends(current_user)):
    conv = DB.get(cid)
    if conv is None or conv["owner"] != user.id:     # same response for 'missing' and 'not yours'
        raise HTTPException(status_code=404, detail="not found")
    return conv
```

◆ **REAL-WORLD PATTERN**

Authorization flaws (BOLA / IDOR) have been the #1 API risk in OWASP's API Top 10 in both the 2019 and 2023 editions. They are simple to explain, easy to prove with two accounts, and often high impact.

3. Practice resources

PortSwigger: Access control portswigger.net/web-security/access-control	**PortSwigger: Authentication** portswigger.net/web-security/authentication	**OWASP API Top 10** owasp.org/API-Security/
OWASP Authorization Cheat Sheet cheatsheetseries.owasp.org	**jwt.io (decode your own tokens)** jwt.io	**OWASP GenAI** genai.owasp.org

★ TAKEAWAY

Logged in ≠ allowed. Test every object and every function with at least two users, prove impact with harmless canaries, and make sure access decisions live in server code — never in the prompt.

PART 4

Hands-on Practical Worksheet

LAB	TIME	ENVIRONMENT
Add auth to your own API and run a two-account authorization test	75–90 minutes	Your Day 4 FastAPI app + curl, localhost only (127.0.0.1)

Task 1 — Add simple token auth to your own API

Start from your Day 4 app (fixed version). Add two demo users with fixed tokens (lab only — real apps use proper login).

● app.py — auth additions

```
# auth part of app.py  (lab only: hard-coded demo tokens)
from fastapi import Depends, Header, HTTPException
TOKENS = {"token-alice": "alice", "token-bob": "bob"}

def current_user(authorization: str = Header(default="")):
    token = authorization.removeprefix("Bearer ").strip()
    if token not in TOKENS:
        raise HTTPException(401, "login required")
    return TOKENS[token]

@app.post("/api/conversations", status_code=201)
def create(data: ConversationIn, user: str = Depends(current_user)):
    conv = {"id": NEXT["id"], "owner": user, "title": data.title, "messages": []}
    DB[conv["id"]] = conv; NEXT["id"] += 1
    return conv

@app.get("/api/conversations/{cid}")
def read(cid: int, user: str = Depends(current_user)):
    if cid not in DB:
        raise HTTPException(404, "not found")
    return DB[cid]                      # TODO for Task 3: is something missing here?
```

Task 2 — Check authentication

Use `curl -i` with and without the header.

Request	Status	Expected?
`GET /api/conversations/1` **with no token**		
Same request with `Authorization: Bearer wrong`		
`POST /api/conversations` **as alice (**`token-alice`**)**		

Task 3 — Two-account authorization test

As alice, create a conversation titled `CANARY-alice-06`. Then request it with bob's token.

Cross-user test

Alice's conversation id: _____

Request made as bob: _____

Status + body bob received: _____

Canary visible to bob? (yes / no): _____

Weakness name + OWASP id: _____

Impact in a real AI app: _____

Task 4 — Fix and retest

Add the ownership check (see Part 3 pattern), also for delete and add-message routes.

Fix + retest

Code change: _____

Bob's request after fix (status): _____

Alice's request after fix (status): _____

Routes you fixed: _____

Finding closed? Evidence: _____

Task 5 — Threat-map one AI feature

Pick one AI feature you use (chat history, file upload, shared workspace). Fill the map without testing it.

Authorization threat map

Feature: _____

Protected resource: _____

Who should access it: _____

Risk hypothesis: _____

How you would test (authorized): _____

Expected secure behavior: _____

⚠ **HONESTY RULE**

If you have not run a test, write **Not tested yet** in the evidence and retest fields. Do not invent results. Real red teamers are trusted because their evidence is real.

Task 6 — Self-check quiz

Answer without looking, then check the key at the end.

Q1. In one line each: what is authentication and what is authorization?

Q2. User B gets User A's conversation by changing `/conversations/1` to `/conversations/2`. What is this weakness called?

Q3. Why must authorization never be decided by the LLM itself?

Q4. Name three things a server must check when it receives a JWT.

Q5. Why do you need at least two test accounts for authorization testing?

Task 7 — End-of-day checklist

- [ ] Finished all three hours (learn, build, red team).
- [ ] Ran the lab myself — not just read about it.
- [ ] Added token auth with two demo users to my own local API.
- [ ] Proved (or disproved) cross-user access with a canary value.
- [ ] Added ownership checks to every route and retested with both users.
- [ ] Separated assumptions from confirmed findings.
- [ ] Recorded evidence honestly (no invented results).
- [ ] Wrote my `DAY-XX.md` notes and pushed them to my notes repo / folder.
- [ ] Ticked today in the 90-Day tracker.

● **DAY-06.md (copy into your notes repo)**

```
# DAY-06.md — Authentication & Authorization
Date:            ____ / ____ / ________
Time spent:      Learn __ min | Build __ min | Red team __ min

## 3 things I learned
1.
2.
3.

## What I built / ran (commands, code, screenshots)
-

## Findings (hypothesis -> test -> evidence -> fix -> retest)
-

## Doubts to research tomorrow
-
```

✓ **ANSWER KEY — CHECK ONLY AFTER YOU TRY**

Q1. Authentication = proving who you are. Authorization = deciding what you are allowed to do or see.

Q2. Broken Object Level Authorization (BOLA), also called IDOR — OWASP API1.

Q3. Model output can be influenced by untrusted input (prompt injection) and is probabilistic. Access decisions must be enforced in trusted server code.

Q4. Valid signature with the expected algorithm and key, not expired (`exp`), correct issuer / audience — and then still check permissions for each resource.

Q5. To check that one user cannot access or change another user's data, and that a normal user cannot use admin functions.